Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

# Отчет по лабораторной работе №2

# «Таблицы»

**Выполнил**:
студент группы 3824Б1ФИ2
Старостин Д.Д.

**Проверил**:
Ивлев А.Д.

Нижний Новгород
2026

# Содержание

Введение ..... 3
1. Постановка задачи ..... 4
2. Руководство пользователя ..... 5
3. Руководство программиста ..... 6
3.1. Описание структуры программы ..... 6
3.2. Описание структур данных ..... 7
3.3. Описание алгоритмов ..... 10
4. Результаты экспериментов и тестирования ..... 11
Заключение ..... 20
Литература ..... 21
Приложение ..... 22

## Введение

В современных информационных системах эффективная организация хранения и обработки данных является одной из ключевых задач. Особое значение имеют структуры данных, обеспечивающие быстрый доступ, вставку и удаление элементов по ключу. К таким структурам относятся различные реализации таблиц: неупорядоченные таблицы, хеш-таблицы и сбалансированные деревья поиска.

Разные подходы к организации таблиц обладают своими преимуществами и недостатками с точки зрения сложности операций, затрат памяти и устойчивости к различным сценариям использования. Поэтому важной задачей является изучение и сравнение эффективности этих структур при выполнении базовых операций.

В данной работе рассматриваются различные способы реализации таблиц и анализируется их поведение при выполнении типовых операций над данными.

# 1. Постановка задачи

**Цель работы** — разработать программу, которая предоставляет возможность хранить данные в таблицах трёх типов с подсчётом количества произведённых операций с момента создания:

1) Неупорядоченная таблица на векторе

2) Упорядоченная таблица на сбалансированном бинарном дереве поиска, а именно на красно-чёрном дереве

3) Хэш-таблица

Таблицы должны поддерживать хранение произвольных данных. В качестве данных в пользовательском консольном приложении, при тестировании и проведении экспериментов необходимо использовать полиномы из Блока 1.

Работоспособность программы необходимо проверить с помощью Google Test. Кроме того, необходимо разработать пользовательское консольное приложение.

**Особенности реализации:**

1. Работа происходит сразу с таблицами всех типов.
2. Ключ – строка, сравнение лексикографическое.
3. Хэш-таблица должна быть реализована с одним из способов борьбы с коллизиями. Хэш-функция любая удовлетворяющая условиям.
4. Пользователь может положить в таблицу свои данные, с заданием ему некоторого наименования (ключа).
5. Пользователь может запросить (выдать или извлечь) из таблицы данные с заданным наименованием (ключом).
6. В пользовательском консольном приложении должна быть возможность производить операции с полиномами, а также размещать полученный результат в таблицу.
7. Во время работы пользовательского консольного приложения происходит логирование количества произведенных операций. После завершения работы в файл добавляется общее число операций для каждого действия и в целом у разных типов таблиц.

## 2. Руководство пользователя

При запуске приложения пользователя встречает меню. С помощью меню можно производить операции с полиномами из списка добавленных пользователем, взаимодействовать со списком полиномов и таблицами, выйти из программы. Взаимодействие с меню происходит посредством ввода числа, ассоциированного с одним из пунктов меню. При неверном вводе выводится ошибка и предоставляется возможность повторного ввода.

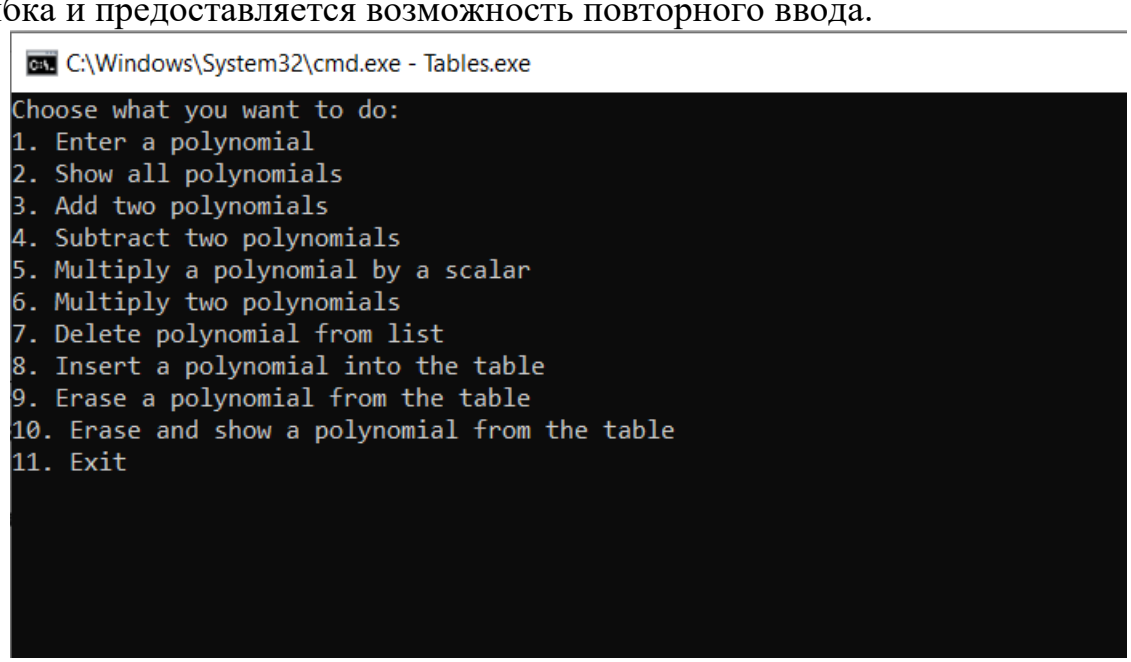


Рис. 1 Меню

**Описание пунктов меню:**

Пункты 1-7 и 11 почти полностью повторяют функционал из лабораторной №1.

**Новый функционал:** в пунктах 3, 4, 5, 6 после получения результата есть возможность добавить его в таблицу, используя в качестве ключа строковое представление полинома или придуманный ключ-строку.

8. «Insert a polynomial into the table». Добавить один из сохранённых в программе полиномов в таблицу. При выборе пункта программа предлагает вывести список доступных полиномов. Далее пользователь должен выбрать номер полинома для вставки и либо использовать в качестве ключа строковое представление полинома, либо придумать свой ключ.

9. «Erase a polynomial from the table». Удалить полином из таблицы. При выборе пункта программа предлагает вывести список существующих ключей. Далее пользователь должен выбрать ключ, запись с которым будет удалена.

10. «Erase and show a polynomial from the table». Извлечь полином из таблицы с выводом на экран. При выборе пункта программа предлагает вывести список существующих ключей. Далее пользователь должен выбрать ключ, запись с которым будет удалена.

# 3. Руководство программиста

## 3.1. Описание структуры программы

Программа состоит из набора модулей, реализующих разные типы таблиц и приложение для работы с ними. Реализовано 3 вида таблиц: неупорядоченная таблица, красно-чёрное дерево и хэш-таблица. Основной функционал: поиск данных по ключу-строке, вставка в таблицу, удаление и извлечение.

Далее описаны модули, представленные в программе.

**Модуль Tables.h**

Базовый абстрактный шаблонный класс Table<T> для таблиц. Содержит счётчики для основных операций, значения которых записываются в файл при уничтожении объекта. В классе объявлены абстрактные методы для последующей реализации в классах-наследниках, а также реализация метода extract.

Чисто виртуальные методы:

- find. Возвращает указатель на данные.
- insert. Вставка записи с заданным ключом-строкой.
- erase. Удаление записи из таблицы.

**Модуль unordered_table.h**

Модуль содержит реализацию неупорядоченной таблицы, класс unordered_table<T>. В таблице хранятся объекты структуры Record. Структура Record хранит ключ и данные.

В классе переопределён метод базового класса extract с целью повышения производительности.

**Модуль hash_table.h**

Модуль содержит реализацию хеш-таблицы, класс hash_table<T>, с цепочками на основе массивов. Особенности данной реализации:

- Ёмкость таблицы ограничена и задаётся при создании объекта. Значение по умолчанию – 100.
- Хэш функция – сумма символов ключа-строки взятая по модулю ёмкости.
- Записи хранятся в структуре Node. Структура содержит ключ, данные и индекс следующего узла цепочки в индексном массиве.

**Модуль rb_tree.h**

Модуль содержит реализацию красно-чёрного дерева, класс rb_tree<T>. Записи хранятся в структуре Node. Структура содержит ключ, данные, цвет узла и указатели на его родителя, левого и правого потомков.

В дереве реализован узел NIL, не содержащий данных и имеющий чёрный цвет. Он используется как фиктивный листовой узел.

Для поддержки сбалансированности дерева реализованы служебные методы leftRotate, rightRotate, insertFixup, deleteFixup, transplant, successor.

Реализованы методы для очистки дерева и проверки его на корректность.

**Модуль user.cpp**

Модуль содержит реализацию консольного приложения. В нём реализованы меню, ввод данных и вывод на экран полиномов, арифметические операции с ними, а также возможность сохранять и извлекать их из таблиц.

**Модуль test.cpp**

Для проверки корректности работы программы был разработан отдельный модуль тестирования. Тестирование реализовано с использованием библиотеки Google Test.

Модуль тестирования содержит набор тестов, проверяющих корректность работы таблиц.

**Модуль experiments.cpp**

Для сравнения эффективности операций вставки, поиска и удаления элементов в таблицах при увеличении объёма данных в данном модуле были проведены эксперименты. Были сгенерированы наборы полиномов размера 100, 1000, 10000, 100000 и для каждого из них фиксировалось количество операций и время их выполнения.

**Модуль полиномов (Polynomial.h, Polynomial.cpp)**

В модуле реализована программная реализация математического полинома, выполненная в ходе лабораторной работы №1.

## 3.2. Описание структур данных

**Класс Table<T>**

Класс Table<T> представляет интерфейс для реализации таблиц на его основе.

*Поля класса:*

- **std::string tableType.** Тип таблицы. Используется вывода.
- **std::ofstream& file.** Ссылка на объект файла для вывода. Передаётся в конструкторе.
- **size_t findOperationsCnt, insertOperationsCnt, eraseOperationsCnt, extractOperationsCnt.** Счётчики для основных операций. Имеют значение по умолчанию 0.

*Методы класса:*

- **Table(std::ofstream& file).** Конструктор. Присваивает полю **file** ссылку на объект файла для вывода.
- **virtual T* find(const std::string& key) = 0.** Чисто виртуальный метод для поиска записи.
- **virtual void insert(const std::string& key, const T& data) = 0.** Чисто виртуальный метод для вставки записи.

- **virtual void erase(const std::string& key) = 0.** Чисто виртуальный метод для удаления записи.
- **virtual bool extract(const std::string& key, T& out).** Виртуальны метод для извлечения. Проверяет наличие записи в таблице с помощью *find* и, если находит – удаляет с помощью *erase* и возвращает true, иначе - false. Данные записи сохраняются в переданную по ссылке переменную.
- **size_t getFindOperationsCount(), size_t getInsertOperationsCnt(), size_t getEraseOperationsCnt(), size_t getExtractOperationsCnt().** Методы для получения значения счётчиков операций.
- **void resetCounters().** Сбрасывает счётчики.
- **std::string getType().** Возвращает строку – тип таблицы.
- **virtual ~Table().** Выводит в файл статистику, созданную за время существования таблицы.

**Класс unordered_table<T>**

Публично наследуется от Table<T>. Реализует неупорядоченную таблицу на векторе.

*Вложенная структура Record:*

Структура представляет запись в таблице.

- Поля структуры:
  - **std::string key.** Ключ.
  - **T data.** Данные.
- Методы структуры:
  - **Record(const std::string key, const T& data).** Конструктор. Задаёт ключ и данные.

*Поля класса:*

- **std::vector<Record> table.** Массив с записями

*Методы класса:*

- **unordered_table(const size_t capacity, std::ofstream& file).** Конструктор с заданием ёмкости для увеличения эффективности.
- **unordered_table(std::ofstream& file) : unordered_table(100, file).** Конструктор без указания ёмкости. Задаёт ёмкость по-умолчанию 100.
- Методы **T* find(const std::string& key) override, void insert(const std::string& key, const T& data) override, void erase(const std::string& key) override** переопределяют методы из базового класса с сохранением изначального назначения.
- **bool extract(const std::string& key, T& out) override.** Извлечение записи. Переопределён для улучшения производительности за счёт уменьшения количества операций.

**Класс hash_table<T>**

Публично наследуется от Table<T>. Реализует хэш-таблицу с цепочками на основе массивов.

*Вложенная структура Node:*

Структура представляет одновременно запись в таблице и узел в односвязном списке.

- Поля структуры:
  - **std::string key.** Ключ.
  - **T data.** Данные.
  - **int next.** Индекс следующего узла в индексном массиве. Значение по умолчанию – -1.
- Методы структуры:
  - **Node(const std::string& key, const T& data, int next).** Конструктор. Задаёт ключ, данные и индекс следующего узла в индексном массиве.

*Поля класса:*

- **const size_t CAPACITY.** Ёмкость таблицы, т.е. количество бакетов. Используется при вычислении хэш-функции.
- **size_t size.** Текущее количество записей в таблице.
- **std::vector<int> buckets.** Индексный массив, содержащий индексы начала цепочек в массиве записей. Размер массива фиксирован и задаётся при создании таблицы. Далее – «массив бакетов».
- **std::vector<Node> nodes.** Массив записей. Содержит узлы цепочек.
- **std::vector<int> freeIndices.** Массив, хранящий освобождённые после удаления индексы в массиве записей. Используется для повторного использования освобождённых позиций и уменьшения количества аллокаций.

*Методы класса:*

- **hash_table(size_t capacity, std::ofstream& file).** Конструктор. Инициализирует ёмкость и массив бакетов размера capacity, резервирует в массиве nodes capacity элементов, чтобы уменьшить количество реалокаций для первый capacity записей.
- **hash_table(std::ofstream& file).** Вышеописанный конструктор с аргументом capacity = 100.
- Методы **T* find(const std::string& key) override, void insert(const std::string& key, const T& data) override, void erase(const std::string& key) override** переопределяют методы из базового класса с сохранением изначального назначения.

**Класс rb_tree<T>**

Публично наследуется от Table<T>. Реализует красно-чёрное дерево.

*Вложенная структура Node:*

Структура представляет одновременно запись в таблице и узел в дереве.

- Поля структуры:
  - **std::string key.** Ключ.
  - **bool color.** Цвет вершины. 0 соответствует чёрному, 1 – красному
  - **T data.** Данные.

    - **Node* left, * right, * parent.** Указатели на левый, правый и родительский узел.
- Методы структуры:
    - **Node(const std::string& key, const T& value).** Конструктор. Задаёт ключ, данные, цвет и указатели. Цвет по умолчанию – красный. Значение указателей по умолчанию – NIL.

*Поля класса:*

- **static Node* NIL.** Статическое поле. Указатель на NIL узел.
- **Node* root.** Указатель на корень дерева.

*Методы класса:*

- **static Node* createNIL().** Создаёт и возвращает указатель на NIL-узел.
- **rb_tree(std::ofstream& file) : Table<T>(file).** Если NIL ещё не создан, создаёт его. Присваивает полю root указатель на NIL.
- Методы **T* find(const std::string& key) override, void insert(const std::string& key, const T& data) override, void erase(const std::string& key) override** переопределяют методы из базового класса с сохранением изначального назначения.
- **void clear(Node* x).** Удаляет все узлы из дерева, кроме NIL.
- **bool isValidRBTree().** Проверка того, что выполняются все свойства красно-чёрного дерева.
- **Node* find(Node* x, const std::string& key).** Служебный метод для поиска узлов в дереве по ключу. Если узел не найден, возвращает NIL.
- **Node* findParent(Node* x, const std::string& key).** Служебный метод для поиска родителей узлов в дереве по ключу. Используется для вставки.
- **void leftRotate(Node* x, int& cnt), void rightRotate(Node* x, int& cnt).** Левый и правый повороты.
- **void insertFixup(Node* x).** Восстановление свойств красно-чёрного дерева после вставки узла.
- **void deleteFixup(Node* x).** Восстановление свойств красно-чёрного дерева после удаления узла.
- **Node* successor(Node* x, int& cnt).** Поиск преемника узла x, т.е. узла, которым можно заменить x без нарушения свойств бинарного дерева поиска.
- **bool checkNode(Node* node, int blackCount, int& expectedBlackHeight).** Служебный метод для проверки свойств красно-чёрного дерева, вызывается функцией *isValidRBTree*.

## 3.3. Описание алгоритмов

**Класс unordered_table<T>**

- **T* find(const std::string& key).** Линейный поиск.

*Сложность:* O(n) по времени.

- **void insert(const std::string& key, const T& data).**
  1) Проверка наличия записи в таблице с помощью линейного поиска.
  2) Если запись с заданным ключом существует – данные обновляются.
  3) Если записи с заданным ключом нет в таблице, создаётся новая запись и вставляется в конец. Возможна перепаковка.

  *Сложность:* O(n) по времени.

- **void erase(const std::string& key).**
  1) Проверка наличия записи в таблице с помощью линейного поиска.
  2) Если запись найдена:
     2.1) Она меняется с последним элементом вектора с помощью swap.
     2.2) Последний элемент вектора (искомая запись) удаляется с помощью pop_back.

  *Сложность:* O(n) по времени.

- **bool extract(const std::string& key, T& out).**
  1) Проверка наличия записи в таблице с помощью линейного поиска.
  2) Если запись найдена:
     2.1) Запись сохраняется в переданную переменную.
     2.2) Она меняется с последним элементом вектора с помощью swap.
     2.3) Последний элемент вектора (искомая запись) удаляется с помощью pop_back.

  *Сложность:* O(n) по времени.

**Класс hash_table<T>**

- **size_t Hash(const std::string& s, size_t& cnt).** Хэш функция.
  1) Суммирует все символы ключа в переменную sum.
  2) Возвращает sum по модулю CAPACITY.

  *Сложность:* O(k), где k — длина строки. При ограниченной длине ключа можно считать O(1).

- **T* find(const std::string& key) override.**
  1) Вычисляет хэш функцию для ключа, т.е. находит в каком бакете находится цепочка с заданным ключом.
  2) Ищет в цепочке запись в заданным ключом.
  3) Если запись найдена, возвращает указатель на неё.

  *Сложность:* O(1) в среднем.

- **bool isInTable(const std::string& key, size_t& _hash, int& out, int& prev, size_t& cnt).** Ищет запись в таблице аналогично методу find, но не возвращает указатель на данные. Возвращает true, если запись найдена, false – иначе.
  *Сложность:* O(1) в среднем.

- **void insert(const std::string& key, const T& data).**
  1) Проверяет, есть ли запись в таблице с помощью *isInTable*.
  2) Если запись уже в таблице – обновляет данные.
  3) Если есть свободные индексы в *freeIndices* один из них, обозначим его id, используется для новой записи. Если не свободных индексов, в массив nodes добавляется новый элемент с индексом id.
     3.1) Если цепочка пуста, в массив buckets записывается id на позицию, вычисленную с помощью хэш функции.
     3.2) Иначе запись добавляется в конец цепочки.
  4) Размер хэш-таблицы увеличивается.

  *Сложность:* O(1) в среднем.

- **void erase(const std::string& key).**
  1) Проверяет, есть ли запись в таблице с помощью *isInTable*.
  2) Если запись найдена, индекс записи добавляется в массив *freeIndices*.
  3) Запись удаляется из таблицы
     3.1) Элемент в начале цепочки: обновляется бакет.
     3.2) Элемент внутри цепочки: обновляется next предыдущего.
  4) Размер хэш-таблицы уменьшается.

  *Сложность:* O(1) в среднем.

- **bool extract(const std::string& key, T& out).**
  1) Выполняется поиск элемента (find).
  2) Если найден:
     2.1) Данные копируются во внешнюю переменную.
     2.2) Элемент удаляется из таблицы (erase).

     *Сложность:* O(1) в среднем.

**Класс rb_tree<T>**

В реализованной структуре поддерживаются следующие свойства:

1. Каждый узел имеет цвет: красный или чёрный.
2. Корень дерева является чёрным узлом.
3. Все листья представлены специальным узлом NIL и являются чёрными.
4. Если узел красный, то у него оба узла потомка чёрные.
5. Для каждого узла все пути от него до листьев содержат одинаковое количество чёрных узлов (чёрная высота).

**Алгоритмы:**

- **Node* find(Node* x, const std::string& key)**

  Поиск выполняется аналогично бинарному дереву поиска:
  1) Начинается с корня.
  2) Ключи сравниваются лексикографически.
  3) Осуществляется переход в левое или правое поддерево в зависимости от результата сравнения.

4) Возвращает указатель на узел, если найден, или NIL.

*Сложность:* O(h) = O(log(n)), т.к. дерево сбалансировано и в методе происходит его обход.

- **T* find(const std::string& key).** Поиск, доступный пользователю. Вызывает *Node* find* и возвращает указатель на данные записи, если она найдена, иначе nullptr.

  *Сложность:* O(log(n)) аналогично вышеописанному методу.

- **void insert(const std::string& key, const T& data).**
  1) Находится родитель будущего узла (findParent).
  2) Новый узел добавляется как лист. Новый узел всегда окрашивается в красный.
  3) Восстановление свойств КЧ дерева (insertFixup)

  *Сложность:* O(log(n)).

- **Node* findParent(Node* x, const std::string& key).**
  1) Начинается с корня.
  2) Ключи сравниваются лексикографически.
  3) Осуществляется переход в левое или правое поддерево в зависимости от результата сравнения, пока не найден родитель искомого узла или потомок узла, в который должен быть переход не NIL.
  4) Возвращает указатель на родителя узла, если найден, или NIL.

  *Сложность:* O(h) = O(log(n)), т.к. дерево сбалансировано и в методе происходит его обход.

- **void insertFixup(Node* x).** Если родитель нового узла красный – нарушается свойство дерева.
  Пока родитель активного узла красный рассматриваются шесть случаев, 3 если родитель узла в левом поддереве и 3 если в правом. Рассмотри первые 3:
  - Случай 1: дядя красный
    1) родитель и дядя перекрашиваются в чёрный, дед — в красный
    2) переход вверх по дереву
  - Случай 2: если не 1.
    1) Активным узлом назначается родитель
    2) выполняется левый поворот
    3) переход к случаю 3
  - Случай 3:
    1) родитель перекрашивается в чёрный, дед в красный
    2) выполняется правый поворот вокруг деда

  В конце корень перекрашивается в чёрный.

*Сложность:* в среднем случаем O(h) = O(log(n)), т.к. дерево сбалансировано и в методе активная вершина поднимается вверх по дереву вдоль ветви.

- **void erase(const std::string& key).**
    1) Удаление как в бинарном дереве поиска, рассматриваются случаи:
        1.1) 0 или 1 потомок: узел заменяется своим потомком (transplant)
        1.2) 2 потомка: узел заменяется своим преемником (successor). Преемник – минимальный элемент правого поддерева.
    2) Запоминается цвет удаляемого узла
    3) Восстановление свойств КЧ дерева (deleteFixup)

*Сложность:* O(log(n)). Дерево сбалансировано, в методе выполняется поиск в дереве. Сложность используемых внутри методов не хуже O(log(n)) в среднем.

- **Node* successor(Node* x, int& cnt).** Поиск минимального потомка в правом поддереве.

*Сложность:* O(log(n)). Дерево сбалансировано, в методе выполняется поиск в дереве.

- **void deleteFixup(Node* x).** Исправление КЧ свойств после удаления. Если удалён чёрный узел, возникает «двойная чёрнота».
  Пока активный узел не корень и является чёрным рассматриваются восемь случаев, 4 если узел в левом поддереве и 4 если в правом.
  Рассмотри первые 4:
    - Случай 1: брат красный
        1) Перекраска: брат –> чёрный, родитель –> красный
        2) Левый поворот вокруг родителя.
        3) Обновление брата
        4) переход к другим случаям
    - Случай 2: брат чёрный, оба его потомка чёрные
        1) перекраска брата в красный
        2) родитель становится активным узлом
    - Случай 3: брат чёрный, его правый потомок красный
        1) Перекраска: левый потомок брата –> чёрный, брат –> красный.
        2) Выполняется правый поворот вокруг брата
        3) Обновление брата
        4) Переход к случаю 4
    - Случай 4:
        1) Перекраска: брат получает цвет родителя, родитель –> чёрный, правый потом брата –> красный
        2) Выполняется левый поворот вокруг родителя

3) Текущий узел – корень
4) Конец цикла.

*Сложность:* O(log(n)), так как количество итераций ограничено высотой дерева.

- **void clear(Node* x).** Рекурсивный обход дерева с удалением узлов. *Сложность: Время:* O(n), удаление всех узлов. Память: O(h) = O(log n) (глубина рекурсии).

- **bool isValidRBTree().**
  Проверяет:
  - цвет корня
  - отсутствие подряд идущих красных узлов
  - одинаковую чёрную высоту всех путей

  *Сложность: Время:* O(n), проверка всех узлов. Память: O(h) = O(log n) (глубина рекурсии).

- **bool checkNode(Node* node, int blackCount, int& expectedBlackHeight).** Реализация *isValidRBTree*.
  *Сложность: Время:* O(n), проверка всех узлов. Память: O(h) = O(log n) (глубина рекурсии).

# 4. Результаты экспериментов и тестирования

## 4.1 Тесты

Для проверки корректности работы реализованных классов было проведено модульное тестирование программы. Тестирование выполнялось с использованием библиотеки Google Test, которая позволяет автоматически проверять корректность работы отдельных компонентов программы.

**Краткое описание тестов:**

1. **Неупорядоченная таблица.** 4 теста.
   - Проверка корректности вставки, поиска, удаления и извлечения
   - Проверка того, что при попытке вставить запись с ключом, который уже находится в таблице, данные обновляются.
2. **Хэш-таблица.** 9 тестов.
   - Проверка корректности вставки, поиска, удаления и извлечения
   - Проверка корректного поведения при коллизиях
   - Проверка того, что при попытке вставить запись с ключом, который уже находится в таблице, данные обновляются.
   - Стресс тест: Вставка, удаление и поиск 1000 элементов.
3. **КЧ дерево.** 7 тестов.
   - Проверка корректности вставки, поиска, удаления и извлечения
   - Проверка того, что при попытке вставить запись с ключом, который уже находится в таблице, данные обновляются
   - Проверка корректной работы пустого дерева
   - Проверка сохранения КЧ свойств при вставке 1000 упорядоченных элементов
   - Стресс тест: корректность проверялась с помощью std::map. 10000 случайных операций: вставка, удаление или поиск.

## 4.2 Эксперименты

Цель экспериментов — сравнение эффективности операций вставки, поиска и удаления в структурах данных: неупорядоченной таблице (unordered table), хэш-таблице (hash table) и красно-чёрном дереве (red-black tree) при увеличении объёма данных. Результаты приведены в приложении №1.

**Методика проведения экспериментов**

Для проведения экспериментов были сгенерированы наборы данных различных размеров (100, 1000, 10000, 100000 элементов), состоящие из

полиномов. В качестве ключей использовались строковые представления полиномов, обеспечивающие уникальность элементов.

Для каждого набора выполнялись следующие операции:

- Вставка: последовательное добавление всех элементов в пустую структуру.
- Поиск: поиск каждого элемента из набора.
- Удаление: последовательное удаление всех элементов.

Для каждой операции фиксировались:

- общее время выполнения.
- количество элементарных операций.

**Результаты экспериментов**

**Малые объёмы данных (n = 100)**

При небольшом размере данных различия между структурами практически не проявляются.

- Вставка быстрее всего выполняется в хэш-таблице.
- Поиск показывает сопоставимые результаты для всех структур.
- Удаление также наиболее эффективно в хэш-таблице.

Это объясняется тем, что при малых n влияние асимптотической сложности незначительно, и доминируют константные факторы.

**Средние объёмы (n = 1000)**

С увеличением объёма данных начинают проявляться различия:

- Хэш-таблица демонстрирует наилучшее время для всех операций.
- Красно-чёрное дерево немного уступает, но остаётся эффективным.
- Неупорядоченная таблица начинает значительно проигрывать по времени.

Количество операций в неупорядоченной таблице растёт квадратично, что соответствует суммарной сложности $O(n^2)$ при выполнении n операций (при линейной сложности одной операции $O(n)$).

**Большие объёмы (n = 10000)**

Различия становятся существенно более выраженными:

- Хэш-таблица остаётся лидером по времени выполнения всех операций.
- Красно-чёрное дерево показывает стабильный рост, соответствующий $O(n \log n)$.

- Неупорядоченная таблица демонстрирует резкое увеличение времени выполнения.

Особенно заметен рост числа операций:

- для неупорядоченной таблицы — до сотен миллионов,
- для хэш-таблицы и дерева — на порядки меньше.

**Очень большие объёмы (n = 100000)**

На больших данных отчётливо проявляется асимптотическое поведение структур:

- Красно-чёрное дерево становится быстрее хэш-таблицы для операций вставки и поиска, что может быть связано с неравномерным распределением ключей хэш-функцией и увеличением числа коллизий.
- Хэш-таблица остаётся наиболее эффективной для удаления.
- Неупорядоченная таблица становится крайне неэффективной (время выполнения на порядки выше).

Количество операций:

- для неупорядоченной таблицы достигает астрономических значений ($\sim 10^{14}$), что подтверждает квадратичную сложность.
- для красно-чёрного дерева — порядка миллионов ($O(n \log n)$ для серии из n операций).
- для хэш-таблицы — десятков миллионов, что соответствует линейной суммарной сложности $O(n)$, однако с большим коэффициентом из-за коллизий.

## Вывод

На основании проведённых экспериментов можно сделать следующие выводы:

1. Неупорядоченная таблица:
- имеет линейную сложность операций.
- при увеличении объёма данных демонстрирует квадратичный рост общего времени.
- непригодна для работы с большими объёмами данных.
2. Хэш-таблица:
- показывает наилучшие результаты на малых и средних данных.
- обеспечивает близкую к $O(1)$ сложность операций в среднем.
- эффективность зависит от качества хэш-функции и распределения данных.
3. Красно-чёрное дерево:
- демонстрирует стабильную производительность $O(\log n)$.

- на больших объёмах данных становится конкурентоспособным и даже превосходит хэш-таблицу по некоторым операциям.
- является более устойчивым к особенностям входных данных.

Экспериментальные результаты подтверждают теоретические оценки сложности:

- O(n) для неупорядоченной таблицы.
- O(1) (в среднем) для хэш-таблицы.
- O(log n) для красно-чёрного дерева.

При этом:

- хэш-таблица наиболее эффективна в среднем случае.
- красно-чёрное дерево обеспечивает более предсказуемую производительность.
- неупорядоченная таблица подходит только для малых наборов данных.

## Заключение

В ходе выполнения работы была достигнута поставленная цель — разработана программа, обеспечивающая хранение данных в таблицах трёх типов: неупорядоченной таблице на векторе, хэш-таблице и упорядоченной таблице на основе красно-чёрного дерева. Для каждой структуры реализована поддержка основных операций: вставки, поиска и удаления элементов, а также реализован подсчёт количества выполненных операций с момента создания таблицы.

Программа обеспечивает работу с произвольными данными. В рамках тестирования и проведения экспериментов в качестве данных использовались полиномы, представленные в виде строковых ключей, что позволило проверить корректность и универсальность реализованных структур.

Была разработана система тестирования с использованием Google Test, что позволило подтвердить корректность реализации всех операций. Также создано пользовательское консольное приложение, обеспечивающее взаимодействие с таблицами и выполнение основных операций.

В рамках экспериментальной части были проведены исследования эффективности операций вставки, поиска и удаления для различных объёмов данных. Полученные результаты показали, что:

- **неупорядоченная таблица** обладает линейной сложностью операций и становится неэффективной при больших объёмах данных.
- **хэш-таблица** демонстрирует наилучшую производительность в среднем случае за счёт близкой к O(1) сложности операций.
- **красно-чёрное дерево** обеспечивает стабильную производительность с логарифмической сложностью и показывает высокую эффективность на больших объёмах данных.

Экспериментальные результаты подтвердили теоретические оценки сложности и позволили выявить особенности поведения каждой структуры данных.

Таким образом, все поставленные задачи выполнены: реализованы требуемые структуры данных, обеспечена их проверка с помощью модульного тестирования, разработано пользовательское приложение и проведено сравнительное исследование их эффективности.

## Литература

Список источников, используемых при выполнении работы.

1. Сайт cppreference: https://en.cppreference.com/
2. Алгоритмы: построение и анализ. Томас Кормен, Чарльз Лейзерсон, Рональд Ривест, Клиффорд Штайн.

# Приложение

Приложение №1. Таблица с результатами экспериментов.

| Size | Operation | metrics | Unordered table | Hash table | Red-Black tree | Minimum time |
|---|---|---|---|---|---|---|
| 100 | Insert | Time (seconds) | 0.0004809 | 9.33E-05 | 0.000199 | Hash table |
| | | Operations count | 25850 | 35354 | 5072 | |
| | Find | Time (seconds) | 8.04E-05 | 8.96E-05 | 8.13E-05 | Unordered table |
| | | Operations count | 51100 | 33633 | 10608 | |
| | Erase | Time (seconds) | 0.0001789 | 7.57E-05 | 0.0001398 | Hash table |
| | | Operations count | 13950 | 34972 | 7497 | |
| 1 000 | Insert | Time (seconds) | 0.0042774 | 0.0013244 | 0.0021663 | Hash table |
| | | Operations count | 2499950 | 358583 | 57022 | |
| | Find | Time (seconds) | 0.0016797 | 0.0007892 | 0.0008616 | Hash table |
| | | Operations count | 4993892 | 341161 | 174004 | |
| | Erase | Time (seconds) | 0.0027557 | 0.0007254 | 0.0017836 | Hash table |
| | | Operations count | 1262841 | 351057 | 80983 | |
| 10 000 | Insert | Time (seconds) | 0.157413 | 0.0179763 | 0.0268787 | Hash table |
| | | Operations count | 242372165 | 3830588 | 552699 | |
| | Find | Time (seconds) | 0.120831 | 0.0103333 | 0.011801 | Hash table |
| | | Operations count | 484683422 | 3654469 | 2383743 | |
| | Erase | Time (seconds) | 0.0752857 | 0.0065495 | 0.0197164 | Hash table |
| | | Operations count | 123216266 | 3477550 | 796640 | |
| 100 000 | Insert | Time (seconds) | 10.9913 | 0.607139 | 0.361776 | Red-Black tree |
| | | Operations count | 7.29956E+14 | 63230023 | 5248225 | |
| | Find | Time (seconds) | 9.68624 | 0.511801 | 0.205554 | Red-Black tree |
| | | Operations count | 43526962562 | 61504408 | 30373277 | |
| | Erase | Time (seconds) | 4.57761 | 0.0706787 | 0.285761 | Hash table |
| | | Operations count | 12108882166 | 35091265 | 7565532 | |